Assignment 2: Git & GitHub Fundamentals – Password Checker

- **Group Size:** 3
- **Chosen Theme:** Password Checker
- **Full Names:** Rania Rahoui, Yasmine Boutchiche, Kamelia Magdoline Milha
- **Student IDs:** 161538 , 156257 , 164183
- **Course Codes & Sections:** 3324 02(rania and yasmine) , CSC 3324 01 (kamelia)
- **GitHub Usernames:** rania-rahoui, yasmine2609, Kamelia-Magdoline
- **GitHub Repository URL:** https://github.com/rania-rahoui/GitLab_Assignment2_PasswordChecker
- **GitHub Pages URL:** https://rania-rahoui.github.io/GitLab_Assignment2_PasswordChecker/

1. Project & Role Distribution

We chose the Password Checker theme and followed the 3-member path. The program checks a password against basic rules, gives it a strength score, and flags weak patterns. Each member owned one feature in its own file, so we could work in parallel without touching each other's code.

Member	Feature	File
Rania Rahoui	Rule checks: length of at least 8, uppercase, lowercase, digit, special character	`password_checker/rules.py`
Yasmine Boutchiche	Strength score from 0 to 5 with a label (Weak, Medium, Strong)	`password_checker/scoring.py`
Kamelia Magdoline Milha	Weak pattern detection: common passwords, repeated characters, sequences	`password_checker/patterns.py`

Rania also created the repository, the initial structure, the ruleset and the GitHub Pages setup.

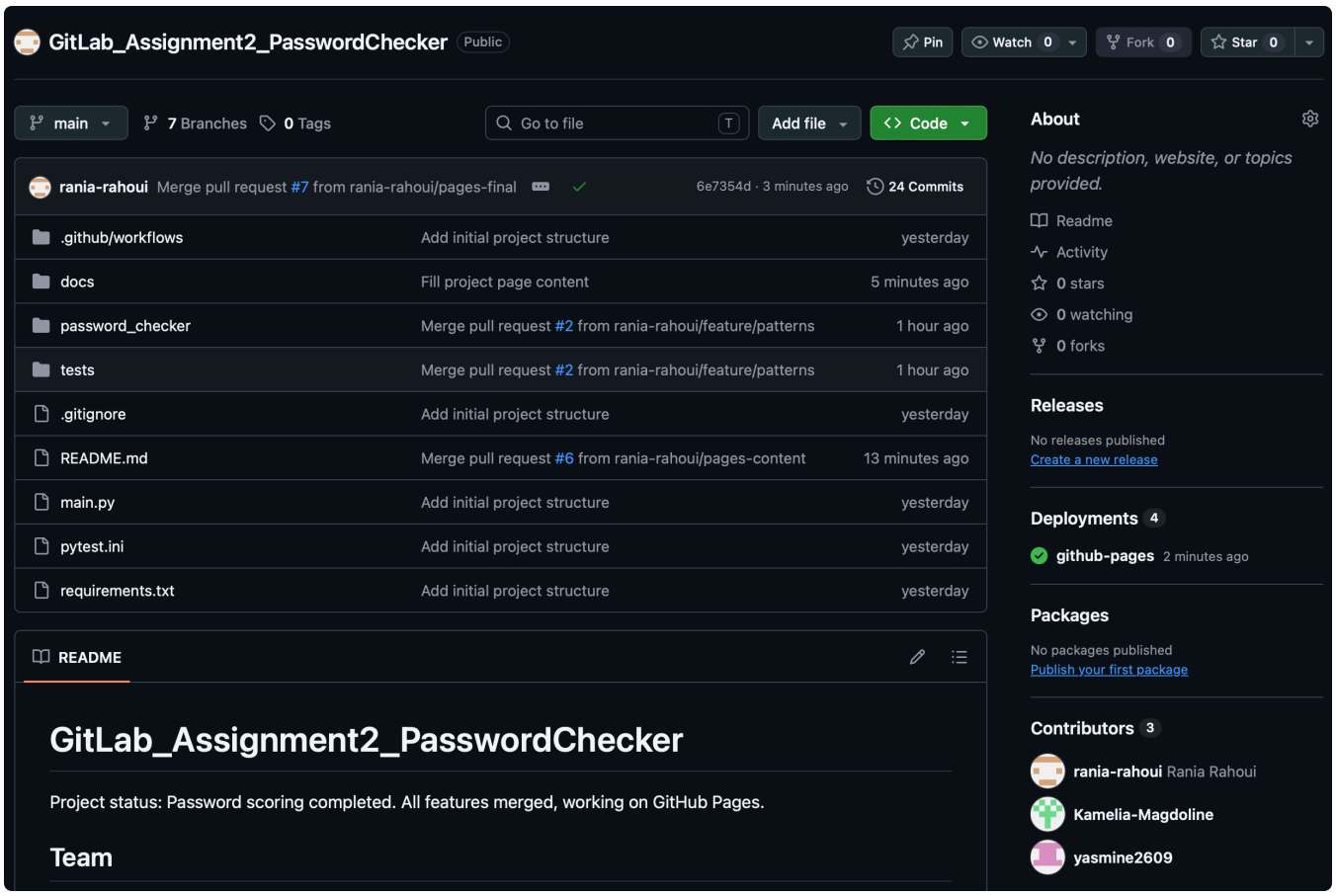


Figure 1. Final repository structure on main, with 3 contributors and a successful Pages deployment.

2. Branches, Commits & Contributions

Nobody developed features directly on main. The only direct push to main was the initial project structure, before the ruleset was turned on.

Member	Main branch	Work completed	Meaningful commits
Rania	`feature/rules`	`check_rules()` returns the list of failed rules, plus 3 tests	"Add initial project structure", "Add password rule checks", "Add unit tests for password rules"
Yasmine	`feature/scoring`	`check_strength()` returns a score from 0 to 5 and a label, plus 4 tests	"Implement password strength scoring", "Add unit tests for password scoring", the Weak label test added after review

Member	Main branch	Work completed	Meaningful commits
Kamelia	`feature/patterns`	Detection of common passwords, repeated characters and sequences, plus 3 tests	"Add weak pattern detection implementation and unit tests", "Trigger Test Failure for CI", "Fix test and pass CI"

Other branches: `status-a` (Rania) and `status-b` (Yasmine) for the merge conflict, and `pages-content` and `pages-final` (Rania) for the project page.

3. Pull Requests & Code Review

Every member authored and reviewed at least one Pull Request. We rotated reviews so nobody reviewed only their own work.

PR	Branch	Author	Reviewer
#1	`feature/rules`	Rania	Kamelia
#2	`feature/patterns`	Kamelia	Yasmine
#3	`feature/scoring`	Yasmine	Rania (Request changes, then approved)
#4	`status-a`	Rania	Yasmine
#5	`status-b`	Yasmine	Kamelia
#6	`pages-content`	Rania	Yasmine
#7	`pages-final`	Rania	Kamelia

Request changes cycle (PR #3). Rania reviewed `scoring.py` in the Files changed tab and noticed that no test covered the "Weak" label, so the `if score <= 2` branch was never tested. She left a line comment on that line asking for a weak password test, and submitted the review as Request changes. Yasmine added `test_weak_password()`, which checks that `check_strength("abc")` returns `(1, "Weak")`. She committed and pushed it to the same `feature/scoring` branch, so the existing PR updated by itself. Rania reviewed again and approved, then the PR was merged.

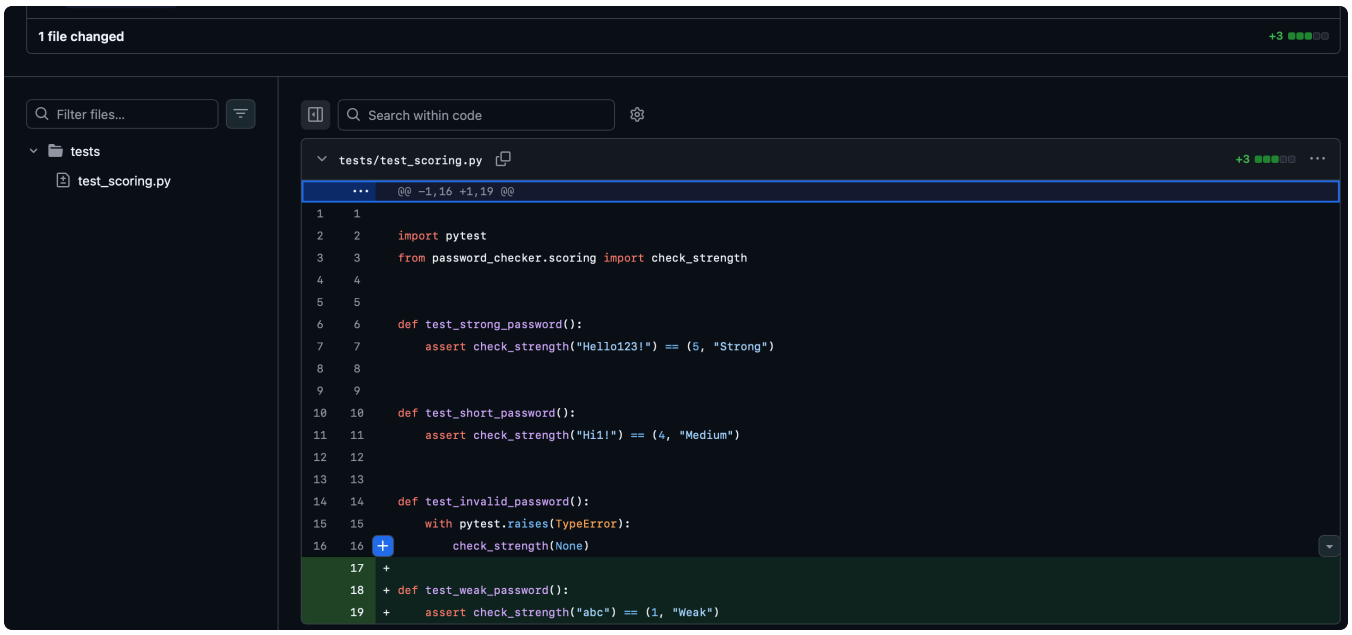


Figure 2. The Weak label test added on `feature/scoring` after the Request changes review.

4. Merge Conflict

After the three features were merged, Rania and Yasmine both created a branch from the same main (`status-a` and `status-b`). Each changed the line `Project status: Development in progress.` in README.md to a different text. Rania's PR #4 was merged first, so main now had her version while `status-b` still had Yasmine's.

On `status-b`, Yasmine ran `git fetch origin` and `git merge origin/main`, and Git reported a conflict in README.md.

- **Current** (between `<<<<<<< HEAD` and `=======`): Yasmine's line from `status-b`.
- **Incoming** (between `=======` and `>>>>>>> origin/main`): Rania's line from main, `All features merged, working on GitHub Pages.`

We decided to keep the information from both lines instead of picking one, since both were true. Yasmine wrote one combined line, removed the conflict markers, then ran `git add README.md`, `git commit` and `git push`. Kamelia reviewed PR #5 and it was merged. The final line is: `Project status: Password scoring completed. All features merged, working on GitHub Pages.`

5. Unit Tests

We used pytest, with one clearly named test file per feature: `test_rules.py`, `test_scoring.py` and `test_patterns.py`. `test_setup.py` only checks that the package imports. In total 11 tests pass locally and in CI.

Case	Example from our tests	Expected result
Normal	`check_rules("StrongPass1!")`	`[]`, all rules pass
Normal	`check_strength("Hello123!")`	`(5, "Strong")`
Edge / boundary	`check_rules("Short1!")`, 7 characters with everything else valid	only the length rule fails
Edge / boundary	`check_strength("abc")`	`(1, "Weak")`, the lowest label
Invalid input	`check_rules(None)` and `check_rules(12345678)`	raises `TypeError`
Invalid input	`check_strength(None)`	raises `TypeError`

Kamelia's `test_patterns.py` checks that common passwords, repeated characters and sequences are flagged.

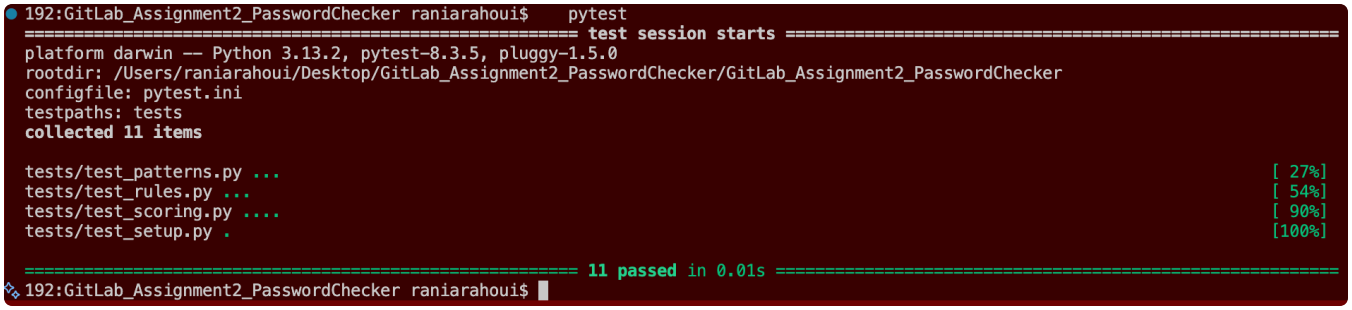


Figure 3. Local pytest run on main after all features were merged: 11 passed.

6. Continuous Integration

Our workflow `.github/workflows/tests.yml` runs on every push and every pull request. It has one job, `test`, on the `ubuntu-latest` runner. Its steps are:

1. `actions/checkout` downloads the repository code onto the runner.
2. `actions/setup-python` installs Python 3.12.

3. `pip install -r requirements.txt` installs pytest.
4. `pytest` runs all the tests.

Failed run. On `feature/patterns`, Kamelia pushed the commit "Trigger Test Failure for CI", which contained a test with a wrong expected value. GitHub Actions marked it with a red X. She opened the failing log, saw which assertion failed, corrected the expected value, and pushed "Fix test and pass CI". The next run passed.

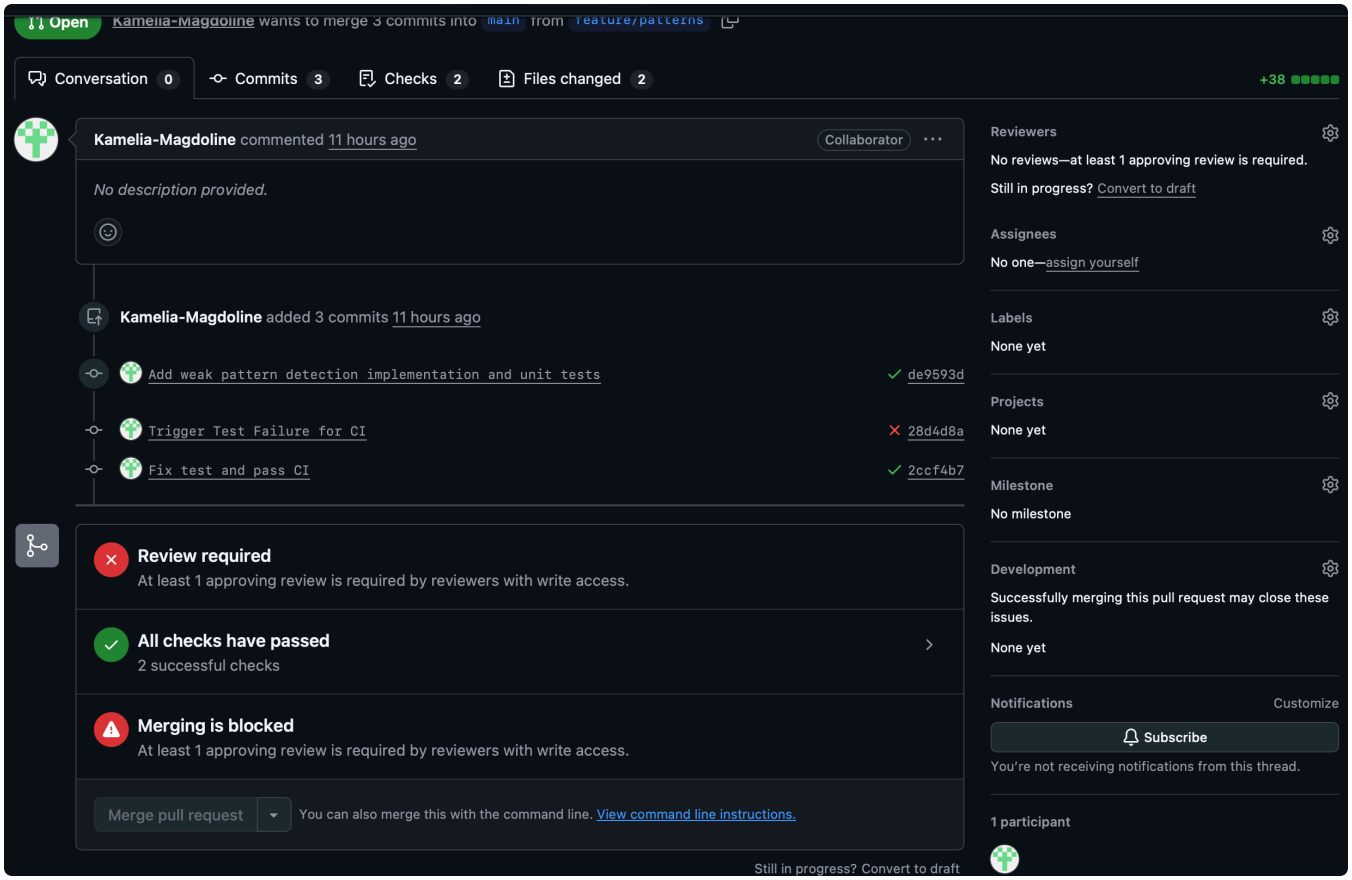


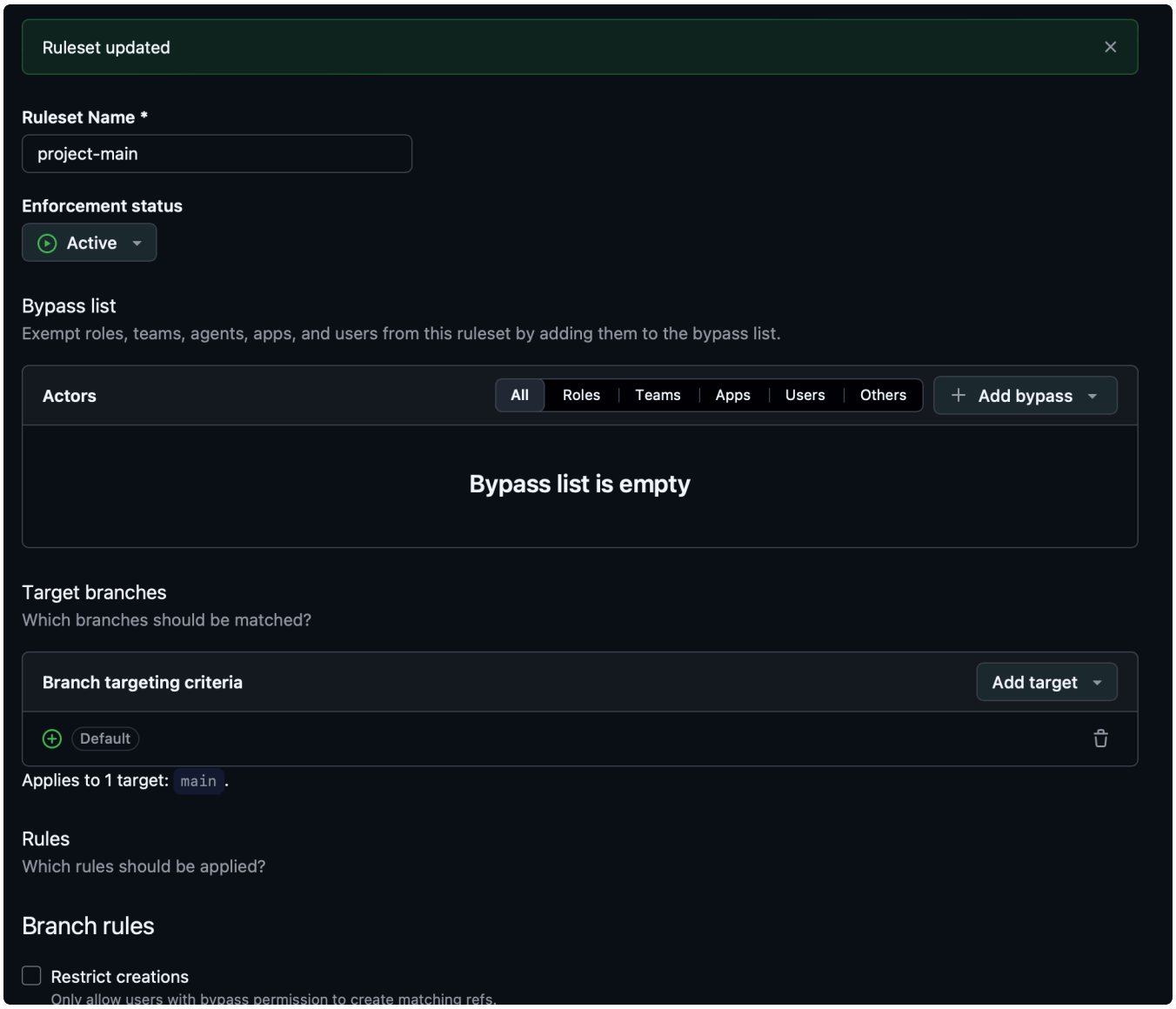
Figure 4. PR #2 commit history: the failed CI run (red X) followed by the fixed, passing run.

7. Ruleset & Protected main

Rania created an active branch ruleset called project-main that targets the default branch (main). The bypass list is empty. The final ruleset:

- Requires a Pull Request before merging, with at least 1 approval
- Requires the `test` status check to pass (added after the workflow had run once and after the failed CI run)
- Restricts branch deletion
- Blocks force pushes

So before any PR could be merged, it needed both an approving review from another member and a green `test` check. Figure 5 shows the final ruleset. Figure 6 shows PR #3 with all checks passed but merging still blocked, because no one had approved it yet.



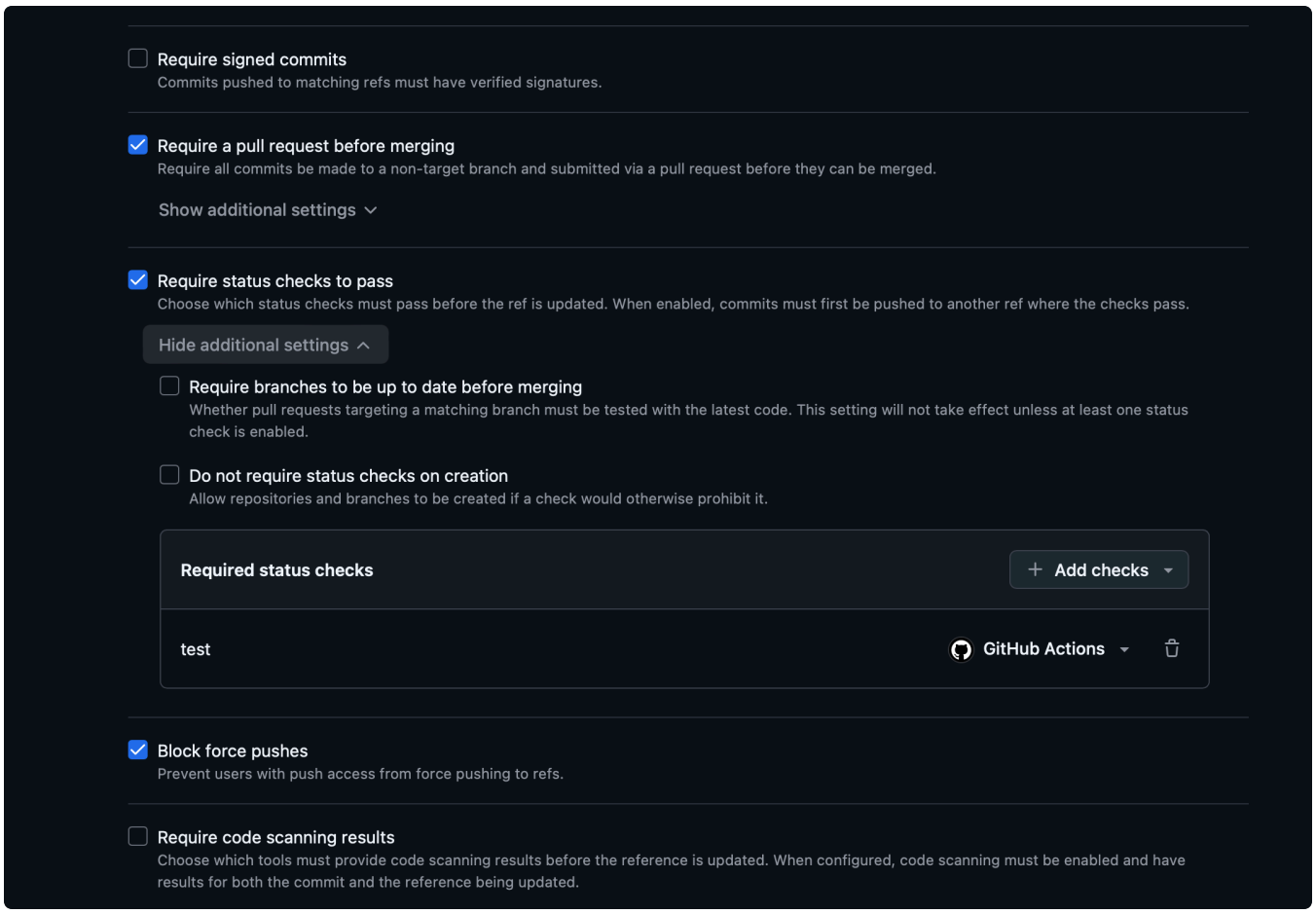


Figure 5. The project-main ruleset: active, targets main, empty bypass list (top); pull request required, `test` status check required (GitHub Actions), force pushes blocked.

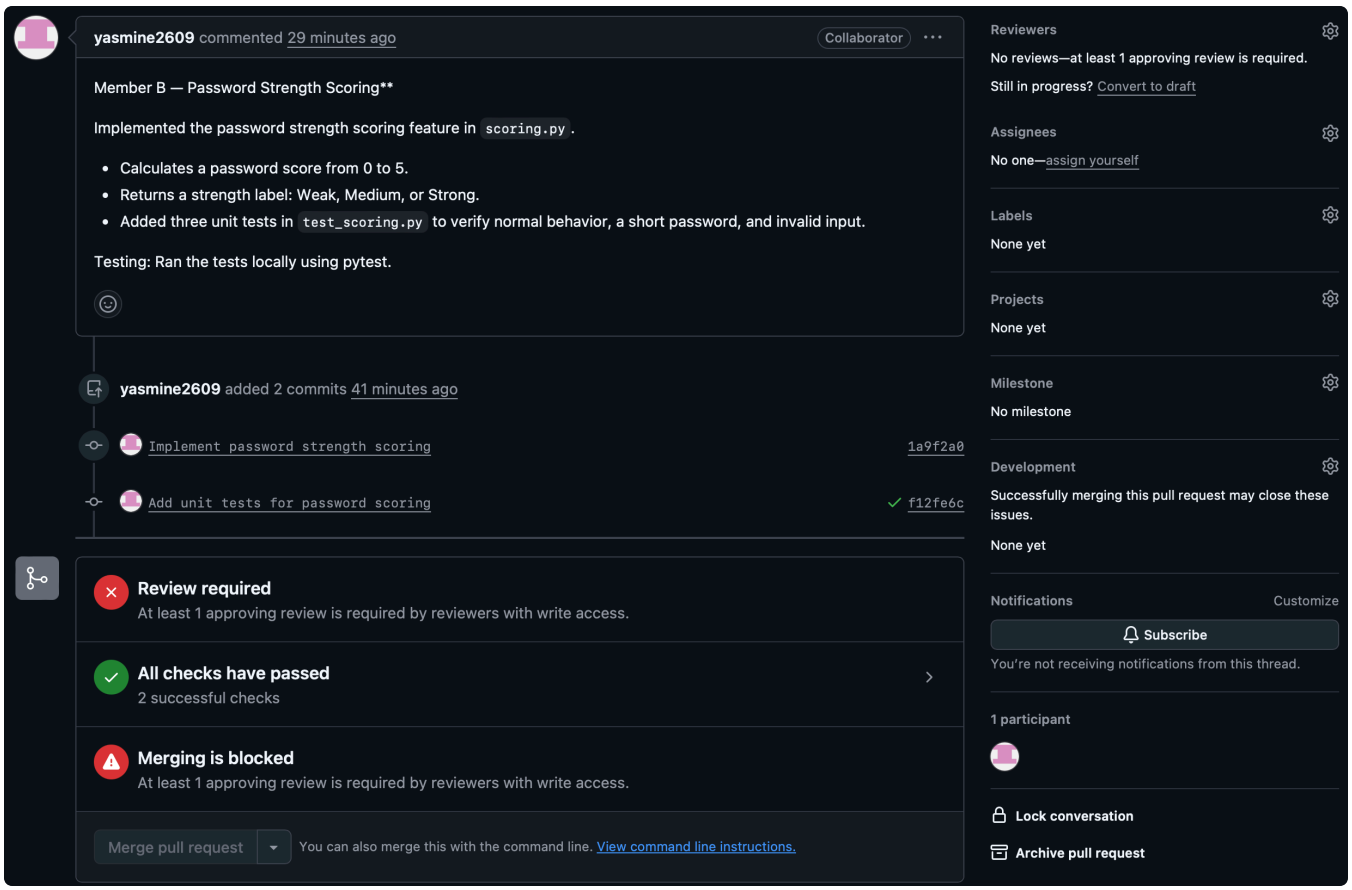


Figure 6. PR #3 blocked: checks passed, but at least 1 approving review is required.

8. Continuous Deployment

GitHub Pages publishes the site from the `/docs` folder on the `main` branch (Settings > Pages > Deploy from a branch). Every merge into main starts the "pages build and deployment" workflow, which republishes `docs/index.html` automatically.

Live site: https://rania-rahoui.github.io/GitLab_Assignment2_PasswordChecker/

Final change. The page first showed placeholder text (`...`) under each heading. Rania replaced it on the `pages-final` branch with the project theme, the team, the features and the run commands, then opened PR #7. After it was approved and merged, a new deployment run finished and the live site showed the new content without any manual step.

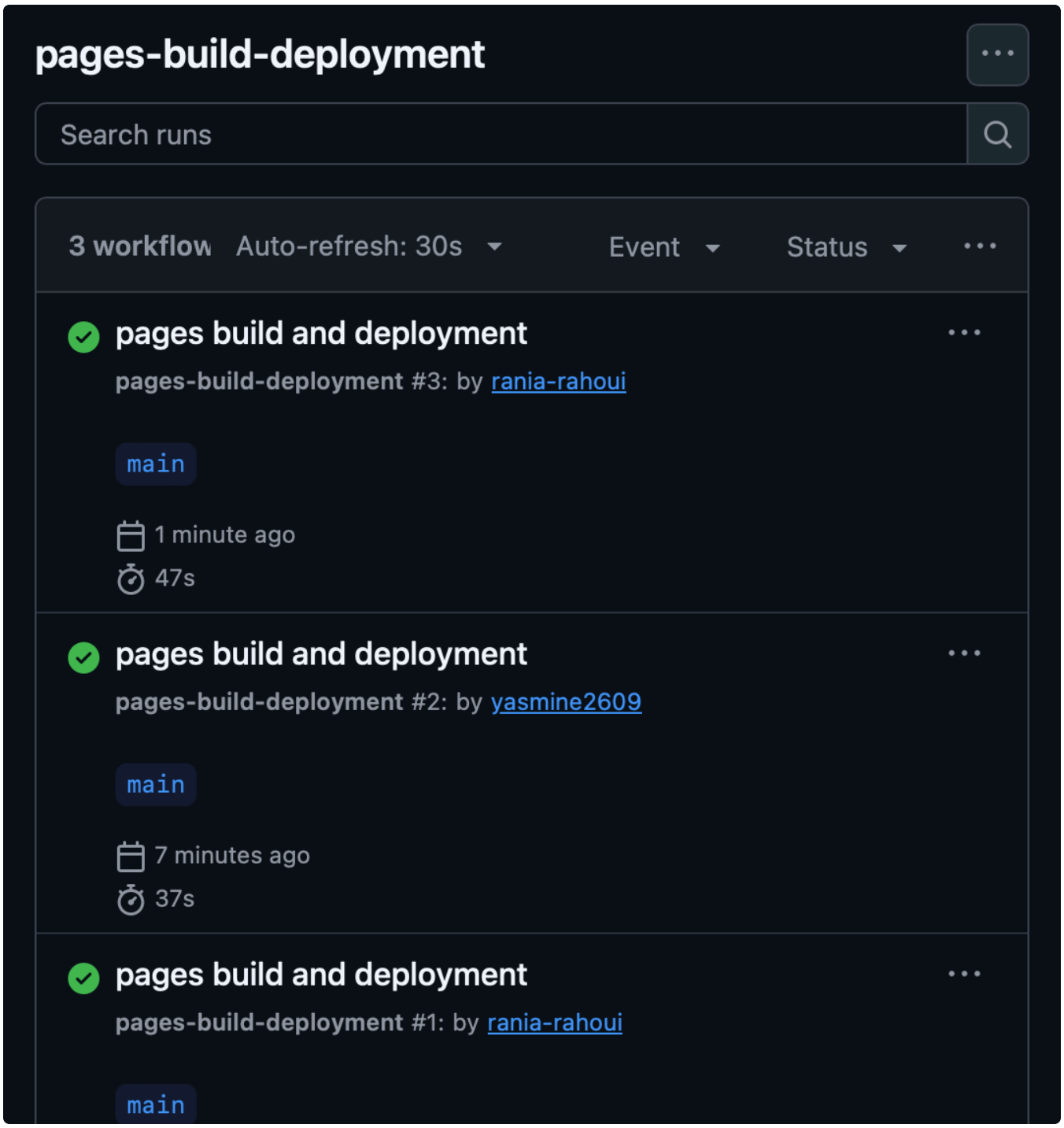


Figure 7. Pages build and deployment runs triggered by merges into main.

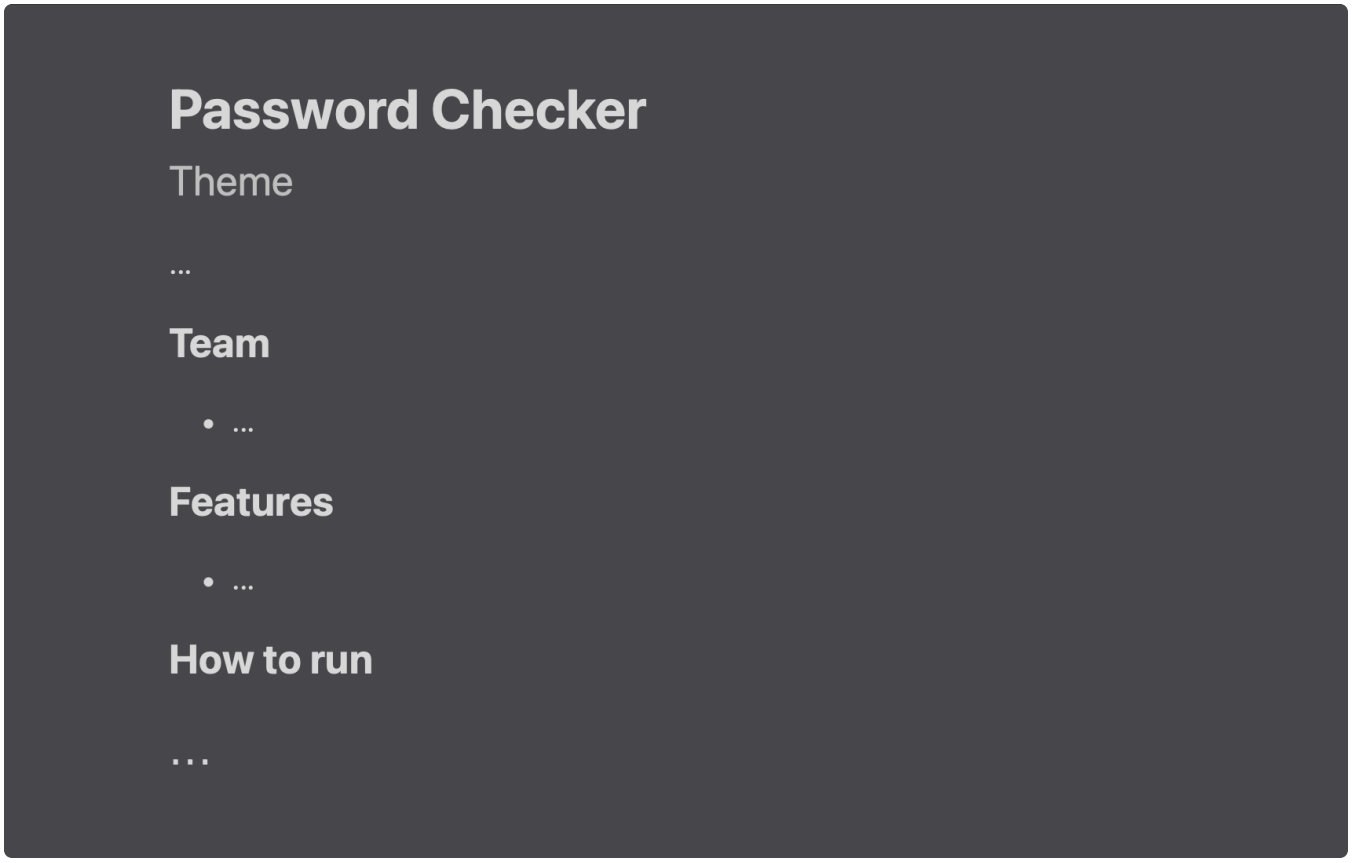


Figure 8. Live site before the final change, with placeholder content.

Password Checker

Theme

Password Checker: checks a password against basic rules, gives it a strength score, and flags weak patterns.

Team

- Rania Rahoui (@rania-rahoui): Rule checks
- Yasmine Boutchiche (@yasmine2609): Strength score
- Kamelia Magdoline (@Kamelia-Magdoline): Weak pattern detection

Features

- Rule checks: length of at least 8, uppercase, lowercase, digit, special character
- Strength score from 0 to 5 with a label (Weak, Medium, Strong)
- Weak pattern detection: common passwords, repeated characters, sequences

How to run

```
pip install -r requirements.txt
pytest
```

Figure 9. Live site after PR #7 was merged.

9. Contribution Summary

Member	Feature(s)	Branch(es)	PRs authored	PRs reviewed	Tests
Rania Rahoui	Rule checks, repo setup, ruleset, GitHub Pages	`feature/rules`, `status-a`, `pages-content`, `pages-final`	#1, #4, #6, #7	#3 (Request changes, then approved)	3 tests in `test_rules.py`
Yasmine Boutchiche	Strength scoring, conflict resolution	`feature/scoring`, `status-b`	#3, #5	#2, #4	4 tests in `test_scoring.py`
Kamelia Magdoline Milha	Weak pattern detection, failed CI run	`feature/patterns`	#2	#1, #5	3 tests in `test_patterns.py`

10. Fair Use of AI

We used Claude during the project. We used it to plan the team split and the order of steps, to explain Git and GitHub concepts, and to fix errors such as a wrong clone URL. Our prompts were mostly questions like "what does this do" or "why does this test fail". We wrote our feature code and tests ourselves and sometimes asked the AI to review them, then fixed the issues it pointed out. The AI also helped with the workflow file, the project page HTML, and organising this report into the template. We checked everything by running `pytest` locally, watching the GitHub Actions results, and opening the live Pages site. Each member understands and is responsible for the code, tests, Git commands and reviews attributed to them.

11. Sources & Reflection

Sources

- GitHub Docs: About rulesets
- GitHub Docs: Workflow syntax for GitHub Actions
- GitHub Docs: Configuring a publishing source for GitHub Pages
- pytest documentation
- Course Session 2 material, and Claude

Reflection

The hardest part was the timing of things. The merge conflict only works if the second branch is created before the first PR is merged, and the `test` status check only appears in the ruleset after the workflow has run once. We also lost time at the start because the clone URL had the wrong username. If a PR, merge, test or workflow behaves unexpectedly, we would first check which branch we are on and run `git pull`, then read the Actions log and run `pytest` locally to see if the problem comes from the code or from the setup.